Advanced machine learning and data analysis for the physical sciences

Morten Hjorth-Jensen[1]

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway[1]

March 19, 2026

© 1999-2026, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Plans for the week March 16-20

1. Discussion of Autoencoders (AEs) and reminder from last week
2. Finalizing discussion of Linear autoencoders and the full connection to PCA
3. Nonlinear and regularized autoencoders and discussion of codes
4. The connection probabilistic autoencoders, PPCA, and VAEs (generative metjods), will be discussed when we start with generative methods (after the holidays)

Our emphasis is on:

1. Linear algebra and optimization,
2. Geometry of latent representations,
3. Reconstruction error minimization,

Reading recommendations

1. Goodfellow et al chapter 14.
2. Rashcka et al. Their chapter 17 contains a brief introduction only.
3. Deep Learning Tutorial on AEs from Stanford University at http://ufldl.stanford.edu/tutorial/unsupervised/Autoencoders/
4. Building AEs in Keras at https://blog.keras.io/building-autoencoders-in-keras.html
5. Introduction to AEs in TensorFlow at https://www.tensorflow.org/tutorials/generative/autoencoder
6. Grosse, University of Toronto, Lecture on AEs at http://www.cs.toronto.edu/~rgrosse/courses/csc321_2017/slides/lec20.pdf
7. Bank et al on AEs at https://arxiv.org/abs/2003.05991
8. Baldi and Hornik, Neural networks and principal component analysis: Learning from examples without local minima, Neural

Autoencoders: Overarching view, reminder from last week

Autoencoders are artificial neural networks capable of learning efficient representations of the input data (these representations are called codings) without any supervision (i.e., the training set is unlabeled). These codings typically have a much lower dimensionality than the input data, making autoencoders useful for dimensionality reduction.
Autoencoders learn to encode the input data into a lower-dimensional representation, and then decode it back to the original data. The goal of autoencoders is to minimize the reconstruction error, which measures how well the output matches the input. Autoencoders can be seen as a way of learning the latent features or hidden structure of the data, and they can be used for data compression, denoising, anomaly detection, and generative modeling.

Powerful detectors

More importantly, autoencoders act as powerful feature detectors, and they can be used for unsupervised pretraining of deep neural networks.

Lastly, they are capable of randomly generating new data that looks very similar to the training data; this is called a generative model. For example, you could train an autoencoder on pictures of faces, and it would then be able to generate new faces. Surprisingly, autoencoders work by simply learning to copy their inputs to their outputs. This may sound like a trivial task, but we will see that constraining the network in various ways can make it rather difficult. For example, you can limit the size of the internal representation, or you can add noise to the inputs and train the network to recover the original inputs. These constraints prevent the autoencoder from trivially copying the inputs directly to the outputs, which forces it to learn efficient ways of representing the data. In short, the codings are byproducts of the autoencoder's attempt to learn the identity function under some constraints.

First introduction of AEs

Autoencoders were first introduced by Rumelhart, Hinton, and Williams in 1986 with the goal of learning to reconstruct the input observations with the lowest error possible.
Why would one want to learn to reconstruct the input observations? If you have problems imagining what that means, think of having a dataset made of images. An autoencoder would be an algorithm that can give as output an image that is as similar as possible to the input one. You may be confused, as there is no apparent reason of doing so. To better understand why autoencoders are useful we need a more informative (although not yet unambiguous) definition.

An autoencoder is a type of algorithm with the primary purpose of learning an "informative" representation of the data that can be used for different applications (see Bank, D., Koenigstein, N., and Giryes, R., Autoencoders) by learning to reconstruct a set of input observations well enough.

Autoencoder structure

Autoencoders are neural networks where the outputs are its own inputs. They are split into an **encoder part** which maps the input $\boldsymbol{x}$ via a function $f(\boldsymbol{x}, \boldsymbol{W})$ (this is the encoder part) to a **so-called code part** (or intermediate part) with the result $\boldsymbol{h}$

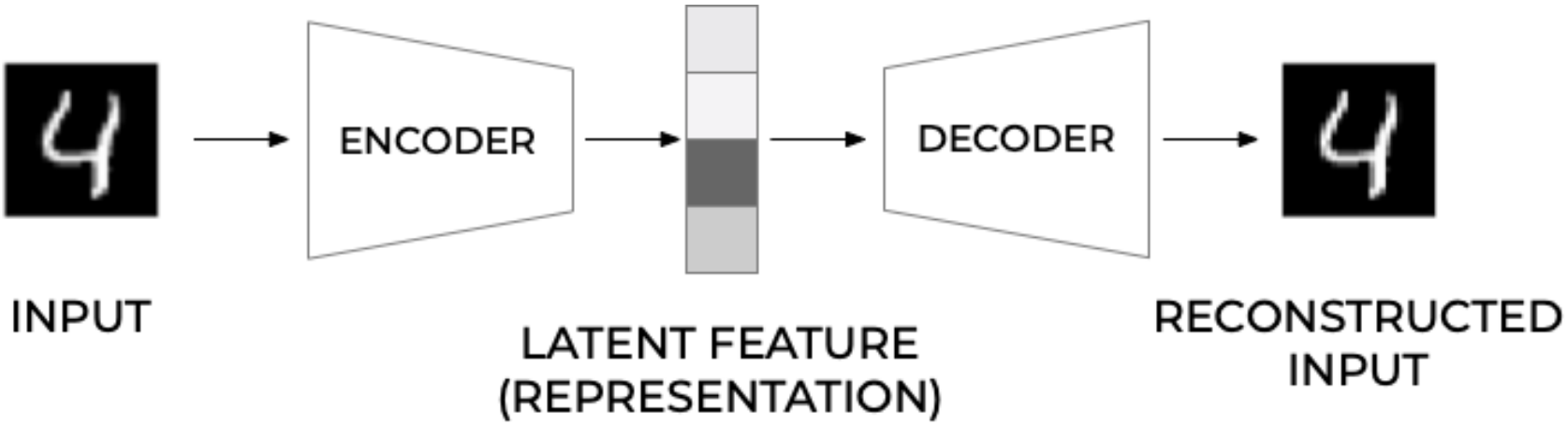
$$\boldsymbol{h} = f(\boldsymbol{x}, \boldsymbol{W})),$$

where $\boldsymbol{W}$ are the weights to be determined. The **decoder** parts maps, via its own parameters (weights given by the matrix $\boldsymbol{V}$ and its own biases) to the final ouput

$$\tilde{\boldsymbol{x}} = g(\boldsymbol{h}, \boldsymbol{V})).$$

The goal is to minimize the construction error.

Schematic image of an Autoencoder



More on the structure

In most typical architectures, the encoder and the decoder are neural networks since they can be easily trained with existing software libraries such as TensorFlow or PyTorch with back propagation.
In general, the encoder can be written as a function g that will depend on some parameters

$$\mathsf{h}_i = g(\mathsf{x}_i),$$

where $\mathsf{h}_i \in \mathbb{R}^q$ (the latent feature representation) is the output of the encoder block where we evaluate it using the input x_i.

Decoder part

Note that we have $g : \mathbb{R}^n \to \mathbb{R}^q$ The decoder and the output of the network $\tilde{\mathrm{x}}_i$ can be written then as a second generic function of the latent features

$$\tilde{\mathrm{x}}_i = f\left(\mathrm{h}_i\right) = f\left(g\left(\mathrm{x}_i\right)\right),$$

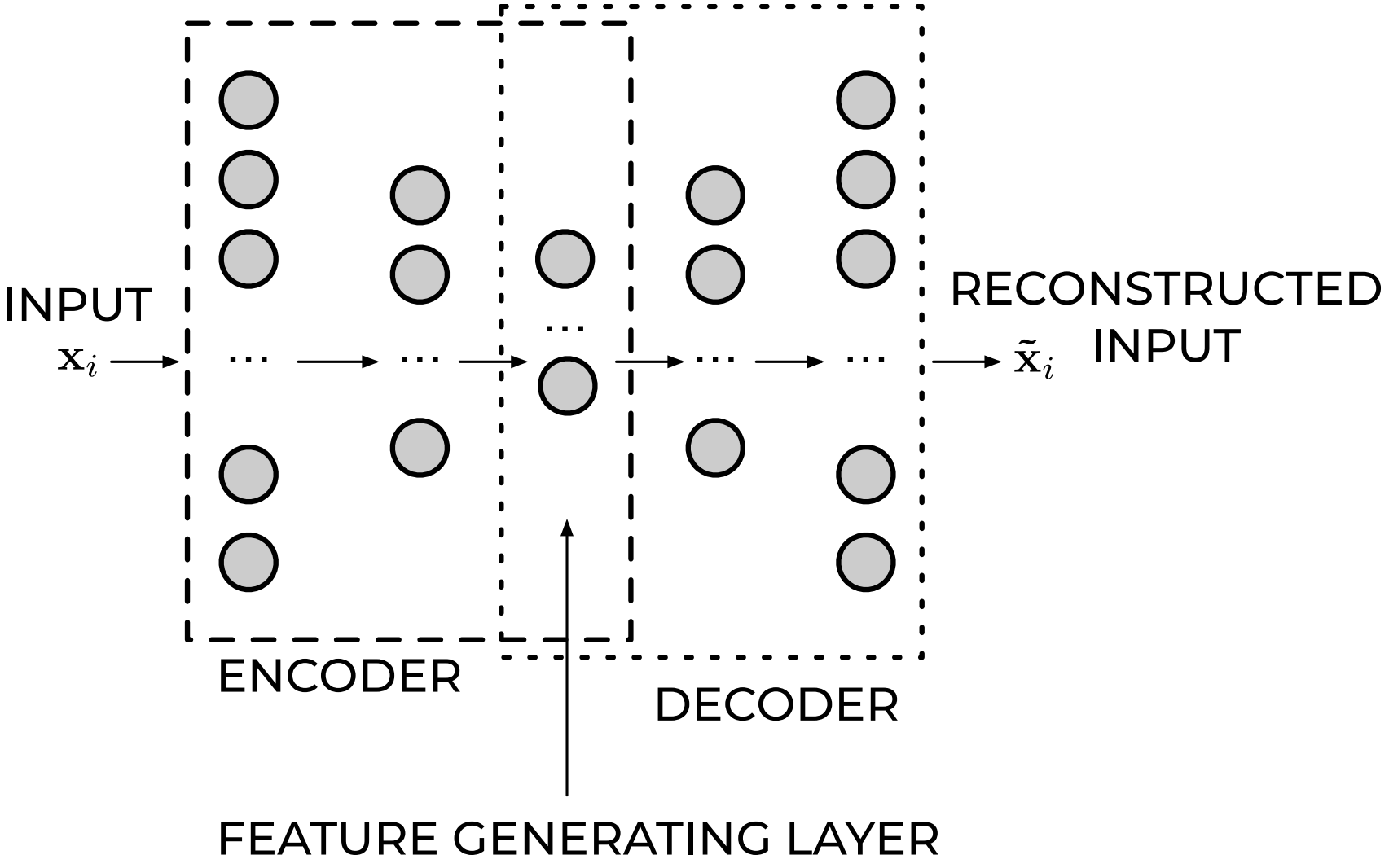
where $\tilde{\mathrm{x}}_i \in \mathbb{R}^n$.
Training an autoencoder simply means finding the functions $g(\cdot)$ and $f(\cdot)$ that satisfy

$$\arg\min_{f,g} < [\Delta(\mathrm{x}_i, f(g\left(\mathrm{x}_i\right))] > .$$

Typical AEs

The standard setup is done via a standard feed forward neural network (FFNN), or what is called a Feed Forward Autoencoder.
A typical FFNN architecture has an odd number of layers and is symmetrical with respect to the middle layer.
Typically, the first layer has a number of neurons $n_1 = n$ which equals the size of the input observation x_i.
As we move toward the center of the network, the number of neurons in each layer drops in some measure. The middle layer usually has the smallest number of neurons. The fact that the number of neurons in this layer is smaller than the size of the input, is often called the **bottleneck**.

Feed Forward Autoencoder



Mirroring

In almost all practical applications, the layers after the middle one are a mirrored version of the layers before the middle one. For example, an autoencoder with three layers could have the following numbers of neurons:
$n_1 = 10$, $n_2 = 5$ and then $n_3 = n_1 = 10$ where the input dimension is equal to ten.
All the layers up to and including the middle one, make what is called the encoder, and all the layers from and including the middle one (up to the output) make what is called the decoder.
If the FFNN training is successful, the result will be a good approximation of the input $\tilde{\mathrm{x}}_i \approx \mathrm{x}_i$.
What is essential to notice is that the decoder can reconstruct the input by using only a much smaller number of features than the input observations initially have.

Output of middle layer

The output of the middle layer h_i are also called a **learned representation** of the input observation x_i.
The encoder can reduce the number of dimensions of the input observation and create a learned representation h_i) of the input that has a smaller dimension $q < n$.
This learned representation is enough for the decoder to reconstruct the input accurately (if the autoencoder training was successful as intended).

Activation Function of the Output Layer

In autoencoders based on neural networks, the output layer's activation function plays a particularly important role. The most used functions are ReLU and Sigmoid.

ReLU

The ReLU activation function can assume all values in the range $[0, \infty]$. As a remainder, its formula is

$$\mathrm{ReLU}\,(x) = \max\,(0, x)\,.$$

This choice is good when the input observations x_i assume a wide range of positive values. If the input x_i can assume negative values, the ReLU is, of course, a terrible choice, and the identity function is a much better choice. It is then common to replace to the ReLU with the so-called **Leaky ReLu** or just modified ReLU.
The ReLU activation function for the output layer is well suited for cases when the input observations x_i assume a wide range of positive real values.

Sigmoid

The sigmoid function σ can assume all values in the range $[0, 1]$,

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

This activation function can only be used if the input observations x_i are all in the range $[0, 1]$ or if you have normalized them to be in that range. Consider as an example the MNIST dataset. Each value of the input observation x_i (one image) is the gray values of the pixels that can assume any value from 0 to 255. Normalizing the data by dividing the pixel values by 255 would make each observation (each image) have only pixel values between 0 and 1. In this case, the sigmoid would be a good choice for the output layer's activation function.

Cost/Loss Function

If an autoencoder is trying to solve a regression problem, the most common choice as a loss function is the Mean Square Error

$$L_{\mathrm{MSE}} = \mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n} ||\mathrm{x}_i - \tilde{\mathrm{x}}_i|\,|_2^2.$$

Reconstruction Error

The reconstruction error (RE) is a metric that gives you an indication of how good (or bad) the autoencoder was able to reconstruct the input observation x_i. The most typical RE used is the MSE

$$\mathrm{RE} \equiv \mathrm{MSE} = \frac{1}{n} \sum_{i=1}^{n} ||\mathrm{x}_i - \tilde{\mathrm{x}}_i| \,|_2^2.$$

Classical PCA Theorem

We assume now that we have a design matrix $\boldsymbol{X}$ which has been centered as discussed above. For the sake of simplicity we skip the overline symbol. The matrix is defined in terms of the various column vectors $[\boldsymbol{x}_0, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_{p-1}]$ each with dimension $\boldsymbol{x} \in \mathbb{R}^n$. The PCA theorem states that minimizing the above reconstruction error corresponds to setting $\boldsymbol{W} = \boldsymbol{S}$, the orthogonal matrix which diagonalizes the empirical covariance(correlation) matrix. The optimal low-dimensional encoding of the data is then given by a set of vectors $\boldsymbol{z}_i$ with at most l vectors, with $l << p$, defined by the orthogonal projection of the data onto the columns spanned by the eigenvectors of the covariance(correlations matrix).

The PCA Theorem

To show the PCA theorem let us start with the assumption that there is one vector $\boldsymbol{s}_0$ which corresponds to a solution which minimized the reconstruction error J. This is an orthogonal vector. It means that we now approximate the reconstruction error in terms of $\boldsymbol{w}_0$ and $\boldsymbol{z}_0$ as

We are almost there, we have obtained a relation between minimizing the reconstruction error and the variance and the covariance matrix. Minimizing the error is equivalent to maximizing the variance of the projected data.

We could trivially maximize the variance of the projection (and thereby minimize the error in the reconstruction function) by letting the norm-2 of $\boldsymbol{w}_0$ go to infinity. However, this norm since we want the matrix $\boldsymbol{W}$ to be an orthogonal matrix, is constrained by $||\boldsymbol{w}_0||_2^2 = 1$. Imposing this condition via a Lagrange multiplier we can then in turn maximize

$$J(\boldsymbol{w}_0) = \boldsymbol{w}_0^T \boldsymbol{C}[\boldsymbol{x}]\boldsymbol{w}_0 + \lambda_0(1 - \boldsymbol{w}_0^T \boldsymbol{w}_0).$$

Taking the derivative with respect to $\boldsymbol{w}_0$ we obtain

Geometric Interpretation and link with Singular Value Decomposition

For a detailed demonstration of the geometric interpretation, see Vidal, Ma and Sastry, section 2.1.2.

Principal Component Analysis (PCA) is by far the most popular dimensionality reduction algorithm. First it identifies the hyperplane that lies closest to the data, and then it projects the data onto it. The following Python code uses NumPy's **svd()** function to obtain all the principal components of the training set, then extracts the first two principal components. First we center the data using either **pandas** or our own code

```
import numpy as np
import pandas as pd
from IPython.display import display
np.random.seed(100)
# setting up a 10 x 5 vanilla matrix
rows = 10
cols = 5
X = np.random.randn(rows,cols)
df = pd.DataFrame(X)
# Pandas does the centering for us
df = df -df.mean()
display(df)
```

PCA and scikit-learn

Scikit-Learn's PCA class implements PCA using SVD decomposition just like we did before. The following code applies PCA to reduce the dimensionality of the dataset down to two dimensions (note that it automatically takes care of centering the data):

```
#thereafter we do a PCA with Scikit-learn
from sklearn.decomposition import PCA
pca = PCA(n_components = 2)
X2D = pca.fit_transform(X)
print(X2D)
```

After fitting the PCA transformer to the dataset, you can access the principal components using the components variable (note that it contains the PCs as horizontal vectors, so, for example, the first principal component is equal to

```
pca.components_.T[:, 0]
```

Another very useful piece of information is the explained variance ratio of each principal component, available via the *explained_variance_ratio* variable. It indicates the proportion of the dataset's variance that lies along the axis of each principal component.

Example of Cancer Data

We can now repeat the above but applied to real data, in this case the Wisconsin breast cancer data. Here we compute performance scores on the training data using logistic regression.

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import  train_test_split
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
cancer = load_breast_cancer()

X_train, X_test, y_train, y_test = train_test_split(cancer.data,cancer

logreg = LogisticRegression()
logreg.fit(X_train, y_train)
print("Train set accuracy from Logistic Regression: {:.2f}".format(log
# We scale the data
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Then perform again a log reg fit
logreg.fit(X_train_scaled, y_train)
print("Train set accuracy scaled data: {:.2f}".format(logreg.score(X_t
#thereafter we do a PCA with Scikit-learn
from sklearn.decomposition import PCA
pca = PCA(n_components = 2)
```

Incremental PCA

One problem with the preceding implementation of PCA is that it requires the whole training set to fit in memory in order for the SVD algorithm to run. Fortunately, Incremental PCA (IPCA) algorithms have been developed: you can split the training set into mini-batches and feed an IPCA algorithm one minibatch at a time. This is useful for large training sets, and also to apply PCA online (i.e., on the fly, as new instances arrive).

Randomized PCA. Scikit-Learn offers yet another option to perform PCA, called Randomized PCA. This is a stochastic algorithm that quickly finds an approximation of the first d principal components. Its computational complexity is $O(m \times d^2) + O(d^3)$, instead of $O(m \times n^2) + O(n^3)$, so it is dramatically faster than the previous algorithms when d is much smaller than n.

Kernel PCA. The kernel trick is a mathematical technique that implicitly maps instances into a very high-dimensional space (called the feature space), enabling nonlinear classification and regression with Support Vector Machines. Recall that a linear decision boundary in the high-dimensional feature space corresponds to a complex nonlinear decision boundary in the original space. It turns

Other techniques

There are many other dimensionality reduction techniques, several of which are available in Scikit-Learn.

Here are some of the most popular:

- **Multidimensional Scaling (MDS)** reduces dimensionality while trying to preserve the distances between the instances.
- **Isomap** creates a graph by connecting each instance to its nearest neighbors, then reduces dimensionality while trying to preserve the geodesic distances between the instances.
- **t-Distributed Stochastic Neighbor Embedding** (t-SNE) reduces dimensionality while trying to keep similar instances close and dissimilar instances apart. It is mostly used for visualization, in particular to visualize clusters of instances in high-dimensional space (e.g., to visualize the MNIST images in 2D).
- Linear Discriminant Analysis (LDA) is actually a classification algorithm, but during training it learns the most discriminative axes between the classes, and these axes can then be used to define a hyperplane onto which to project the data. The benefit is that the projection will keep classes as far apart as

Why go beyond PCA?

PCA captures only linear subspaces. But many datasets lie near nonlinear manifolds:

- images of a rotated object,
- molecular conformations,
- solutions of nonlinear PDEs,
- speech signals.

A nonlinear autoencoder replaces linear maps by compositions of affine maps and nonlinear activations.

Deep autoencoder architecture

A deep encoder may have the form

$$z = f_\theta(x) = \sigma_L(W_L \sigma_{L-1}(W_{L-1} \cdots \sigma_1(W_1 x + b_1) \cdots + b_{L-1}) + b_L),$$

and the decoder similarly

$$\hat{x} = g_\phi(z).$$

These nested nonlinear maps can approximate complicated non-linear structures.

Local linearization of nonlinear autoencoders

If f and g are differentiable, then around x_0,

$$f(x) \approx f(x_0) + J_f(x_0)(x - x_0),$$

$$g(z) \approx g(z_0) + J_g(z_0)(z - z_0).$$

Hence

$$g(f(x)) \approx g(f(x_0)) + J_g(z_0)J_f(x_0)(x - x_0).$$

So the Jacobian product $J_g J_f$ acts as the local linear reconstruction operator.

Denoising autoencoders

In a denoising autoencoder, one corrupts the input:

$$\tilde{x} = x + \varepsilon,$$

and trains the network to reconstruct x from $\tilde{x}$:

$$\min_{\theta,\phi} \frac{1}{N} \sum_{n=1}^{N} \|x_n - g_\phi(f_\theta(\tilde{x}_n))\|_2^2.$$

This prevents the model from trivially copying the input and encourages extraction of robust structure.

Statistical meaning of denoising

For small Gaussian noise, the denoising map approximates a score-like direction. In fact, denoising autoencoders are closely related to score matching: the reconstruction vector

$$r(x) - x$$

encodes information about the gradient of the log-density of the data distribution. Thus denoising autoencoders anticipate ideas that later appear in diffusion and score-based models.

Sparse autoencoders

Sparse autoencoders add a penalty encouraging latent activations to be mostly zero:

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \|x_n - \hat{x}_n\|_2^2 + \lambda \sum_{n=1}^{N} \|z_n\|_1.$$

Interpretation:

- each input activates only a small subset of latent units,
- latent units can become feature detectors,
- the model resembles dictionary learning and sparse coding.

Contractive autoencoders

Contractive autoencoders penalize the sensitivity of the encoder:

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \|x_n - \hat{x}_n\|_2^2 + \lambda \sum_{n=1}^{N} \|J_f(x_n)\|_F^2.$$

This encourages the representation to be stable under small perturbations orthogonal to the data manifold. Hence the encoder becomes approximately invariant to directions that do not matter.

Jacobian penalty and geometry

The Jacobian $J_f(x)$ measures how latent coordinates change with the input. If

$$\|J_f(x)\|_F^2$$

is small in directions orthogonal to the manifold, then the encoder contracts those directions. So contractive autoencoders can be seen as learning tangent directions of the data manifold while suppressing normal directions.

Overcomplete autoencoders and regularization

An overcomplete autoencoder with $k \geq d$ can easily learn

$$g(f(x)) = x.$$

To prevent this, one must impose additional structure:

- sparsity,
- denoising corruption,
- contractive penalty,
- weight decay or architectural constraints.

Thus regularization can replace the role of a bottleneck.

Kernel PCA vs nonlinear autoencoders

Kernel PCA uses a fixed nonlinear feature map $\Phi(x)$ and performs PCA in feature space. A nonlinear autoencoder instead learns both:

- the representation map,
- and the inverse reconstruction map.

Kernel PCA is nonparametric and spectral; autoencoders are parametric and optimization-based.

Optimization landscape

Nonlinear autoencoder training is nonconvex because of:

- multilayer compositions,
- nonlinear activations,
- coupled encoder/decoder parameters.

Thus unlike linear autoencoders, we generally do not have closed-form global minimizers. Instead, gradient-based optimization is used.

Generalization issues

A nonlinear autoencoder can memorize the training set if too expressive. Hence we must ask:

- does it generalize to unseen data?
- is the latent structure stable?
- does reconstruction preserve physically meaningful variation?

These questions motivate probabilistic and information-theoretic viewpoints.

From deterministic to probabilistic models

Deterministic autoencoders produce a code

$$z = f_\theta(x)$$

and a reconstruction

$$\hat{x} = g_\phi(z).$$

A probabilistic autoencoder instead introduces a latent-variable model

$$p_\theta(x, z) = p_\theta(x \mid z)p(z),$$

where z is a random latent variable. This turns representation learning into statistical inference.

Probabilistic PCA model

Probabilistic PCA (PPCA) assumes

$$z \sim \mathcal{N}(0, I_k),$$

and

$$x = Wz + \mu + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2 I_d).$$

Hence

$$p(x \mid z) = \mathcal{N}(Wz + \mu, \sigma^2 I_d).$$

This is a linear-Gaussian latent variable model.

Marginal distribution in PPCA

Integrating out z, one gets

$$x \sim \mathcal{N}(\mu,\, WW^T + \sigma^2 I_d).$$

Thus PPCA models the covariance by a low-rank term plus isotropic noise:

$$\Sigma = WW^T + \sigma^2 I_d.$$

This is the probabilistic analogue of low-dimensional linear structure.

PPCA and classical PCA

As $\sigma^2 \to 0$, the PPCA model approaches a deterministic projection model. Moreover, maximum-likelihood estimation in PPCA yields a loading matrix whose column space is the PCA subspace. So PPCA recovers the same principal directions as classical PCA, but with a probabilistic interpretation.
In our discussions of generative models, we will dive deeper into probabilistic approaches.

Codes for Autoencoders: Linear Autoencoders

The examples here can be easily modified and adapted to different data sets. The first example is a straightforward AE.

```
import sys
assert sys.version_info >= (3, 5)

# Is this notebook running on Colab or Kaggle?
IS_COLAB = "google.colab" in sys.modules
IS_KAGGLE = "kaggle_secrets" in sys.modules

# Scikit-Learn >=0.20 is required
import sklearn
assert sklearn.__version__ >= "0.20"

# TensorFlow >= 2.0 is required
import tensorflow as tf
from tensorflow import keras
assert tf.__version__ >= "2.0"

if not tf.config.list_physical_devices('GPU'):
    print("No GPU was detected. LSTMs and CNNs can be very slow withou
    if IS_COLAB:
        print("Go to Runtime > Change runtime and select a GPU hardwar
    if IS_KAGGLE:
        print("Go to Settings > Accelerator and select GPU.")

# Common imports
import numpy as np
import os
```

More advanced features, stacked AEs

```
# You can select the so-called fashion data as well, here we just use
#(X_train_full, y_train_full), (X_test, y_test) = keras.datasets.fashi
(X_train_full, y_train_full), (X_test, y_test) = keras.datasets.mnist.
X_train_full = X_train_full.astype(np.float32) / 255
X_test = X_test.astype(np.float32) / 255
X_train, X_valid = X_train_full[:-5000], X_train_full[-5000:]
y_train, y_valid = y_train_full[:-5000], y_train_full[-5000:]
```

We can now train all layers at once by building a stacked AE with 3 hidden layers and 1 output layer (i.e., 2 stacked Autoencoders).

```
def rounded_accuracy(y_true, y_pred):
    return keras.metrics.binary_accuracy(tf.round(y_true), tf.round(y_
tf.random.set_seed(42)
np.random.seed(42)

stacked_encoder = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(100, activation="selu"),
    keras.layers.Dense(30, activation="selu"),
])
stacked_decoder = keras.models.Sequential([
    keras.layers.Dense(100, activation="selu", input_shape=[30]),
    keras.layers.Dense(28 * 28, activation="sigmoid"),
    keras.layers.Reshape([28, 28])
])
stacked_ae = keras.models.Sequential([stacked_encoder, stacked_decoder
stacked_ae.compile(loss="binary_crossentropy",
                   optimizer=keras.optimizers.SGD(learning_rate=1.5)
```

Using Convolutional Layers Instead of Dense Layers

```
tf.random.set_seed(42)
np.random.seed(42)

conv_encoder = keras.models.Sequential([
    keras.layers.Reshape([28, 28, 1], input_shape=[28, 28]),
    keras.layers.Conv2D(16, kernel_size=3, padding="SAME", activation=
    keras.layers.MaxPool2D(pool_size=2),
    keras.layers.Conv2D(32, kernel_size=3, padding="SAME", activation=
    keras.layers.MaxPool2D(pool_size=2),
    keras.layers.Conv2D(64, kernel_size=3, padding="SAME", activation=
    keras.layers.MaxPool2D(pool_size=2)
])
conv_decoder = keras.models.Sequential([
    keras.layers.Conv2DTranspose(32, kernel_size=3, strides=2, padding
                                 input_shape=[3, 3, 64]),
    keras.layers.Conv2DTranspose(16, kernel_size=3, strides=2, padding
    keras.layers.Conv2DTranspose(1, kernel_size=3, strides=2, padding=
    keras.layers.Reshape([28, 28])
])
conv_ae = keras.models.Sequential([conv_encoder, conv_decoder])

conv_ae.compile(loss="binary_crossentropy", optimizer=keras.optimizers
                metrics=[rounded_accuracy])
history = conv_ae.fit(X_train, X_train, epochs=5,
                      validation_data=(X_valid, X_valid))

conv_encoder.summary()
conv_decoder.summary()
```

Recurrent Autoencoders

```
recurrent_encoder = keras.models.Sequential([
    keras.layers.LSTM(100, return_sequences=True, input_shape=[28, 28]
    keras.layers.LSTM(30)
])
recurrent_decoder = keras.models.Sequential([
    keras.layers.RepeatVector(28, input_shape=[30]),
    keras.layers.LSTM(100, return_sequences=True),
    keras.layers.TimeDistributed(keras.layers.Dense(28, activation="si
])
recurrent_ae = keras.models.Sequential([recurrent_encoder, recurrent_d
recurrent_ae.compile(loss="binary_crossentropy", optimizer=keras.optim
                     metrics=[rounded_accuracy])
history = recurrent_ae.fit(X_train, X_train, epochs=10, validation_dat

show_reconstructions(recurrent_ae)
plt.show()
```

Stacked denoising Autoencoder with Gaussian noise

```
tf.random.set_seed(42)
np.random.seed(42)

denoising_encoder = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.GaussianNoise(0.2),
    keras.layers.Dense(100, activation="selu"),
    keras.layers.Dense(30, activation="selu")
])
denoising_decoder = keras.models.Sequential([
    keras.layers.Dense(100, activation="selu", input_shape=[30]),
    keras.layers.Dense(28 * 28, activation="sigmoid"),
    keras.layers.Reshape([28, 28])
])
denoising_ae = keras.models.Sequential([denoising_encoder, denoising_d
denoising_ae.compile(loss="binary_crossentropy", optimizer=keras.optim
                     metrics=[rounded_accuracy])
history = denoising_ae.fit(X_train, X_train, epochs=10,
                           validation_data=(X_valid, X_valid))

tf.random.set_seed(42)
np.random.seed(42)

noise = keras.layers.GaussianNoise(0.2)
show_reconstructions(denoising_ae, noise(X_valid, training=True))
plt.show()
```

And using dropout

PyTorch example

We will continue with the MNIST database, which has 60000 training examples and a test set of 10000 handwritten numbers. The images have only one color channel and have a size of 28×28 pixels. We start by uploading the data set.

```
# import the Torch packages
# transforms are used to preprocess the images, e.g. crop, rotate, nor
import torch
from torchvision import datasets,transforms

# specify the data path in which you would like to store the downloade
# ToTensor() here is used to convert data type to tensor

train_dataset = datasets.MNIST(root='./mnist_data/', train=True, trans
test_dataset = datasets.MNIST(root='./mnist_data/', train=False, trans
print(train_dataset)
batchSize=128

#only after packed in DataLoader, can we feed the data into the neural
train_loader = torch.utils.data.DataLoader(dataset=train_dataset, batc
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_
```

We visualize the images here using the *imshow* function function and the *make_grid* function from PyTorch to arrange and display them.

```
# package we used to manipulate matrix
```